

B. R. HARNE COLLEGE OF ENGINEERING & TECHNOLOGY

Karav, Vangani, Tal. Ambernath, Dist. Thane — 421 503

DEPARTMENT OF COMPUTER ENGINEERING

(Affiliated with the University of Mumbai)

CERTIFICATE

This is to certify that the requirements for the synopsis entitled

"CYBERSENTINEL AI: AN AI-POWERED SYSTEM FOR REAL-TIME CYBER ATTACK DETECTION AND AUTOMATED SECURITY RESPONSE"

have been successfully completed by the following students:

1. Soham Sakat

Roll No: _____

2. [Student Name 2]

Roll No: _____

3. [Student Name 3]

Roll No: _____

4. [Student Name 4]

Roll No: _____

in partial fulfillment of Sem - VII, Bachelor of Engineering of Mumbai University in Computer Engineering at B. R. Harne College of Engineering & Technology, Karav, Vangani affiliated with Mumbai University for the academic year 2026-27.

Internal Guide

(Prof. [Name of Guide])

External Examiner

Project Coordinator

(Prof. Vaibhav Dhage)

Head of Department

(Dr. Shital Agrawal)

Principal

(Dr. Vikram Patil)

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included; I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission.

Date: _____

(Name of the Students & Signature)

Place: Karav, Vangani

1. Soham Sakat (_____)
2. [Student Name 2] (_____)
3. [Student Name 3] (_____)
4. [Student Name 4] (_____)

ACKNOWLEDGEMENT

A project is something that could not have been materialized without cooperation of many people. This project shall be incomplete if I do not convey my heartfelt gratitude to those people from whom I have got considerable support and encouragement.

It is a matter of great pleasure for us to have respected **Prof. [Name of Guide]** as our project guide. We are thankful to her for being a constant source of inspiration and guidance throughout the design and development of this work.

We would also like to give our sincere thanks to **Dr. Shital Agrawal**, Head of Department, Computer Engineering, and **Prof. Vaibhav Dhage**, Project Coordinator, for their kind support and continuous encouragement.

We would like to express our deepest gratitude to **Dr. Vikram Patil**, Principal of B. R. Hare College of Engineering & Technology, for providing the necessary institutional facilities.

ABSTRACT

Today, companies and organisations receive thousands of computer security alerts every single day. Out of all these alerts, more than 90% are false alarms — meaning they are not real attacks at all. Security teams have to manually check each alert, which takes a lot of time and effort. Because of this overload, security staff often miss actual cyber attacks. On average, a hacker can stay hidden inside a company's network for more than 200 days before anyone notices.

Existing security tools either use simple fixed rules that fail to catch new types of attacks, or they use general AI tools that sometimes give wrong or made-up answers — which is very dangerous in cybersecurity. This project presents **CyberSentinel AI** — a smart, automated security assistant that reads computer log files, filters out harmless events, identifies real attacks by comparing them with a database of 600+ known hacking techniques, gives each attack a danger score from 0 to 100, and automatically blocks the attacker. The system can detect and respond to a cyber attack in under 2.4 seconds, compared to 45 to 60 minutes when done manually. It also generates a formal PDF report of the incident automatically.

CHAPTER 1: INTRODUCTION

1.1 What is a Security Operations Center (SOC)?

Every large company that uses computers has a team of people whose job is to keep the computer systems safe. This team is called the Security Operations Center (SOC). Their job is to look at logs — records of activity — coming from computers, servers, and networks, and decide if something suspicious is happening, like a hacker trying to break in. These teams work 24 hours a day, 7 days a week, checking alerts and investigating incidents.

1.2 The Problem: Too Many Alerts, Too Little Time

The biggest challenge for security teams today is the huge number of alerts they receive. A typical company can receive between **50,000 and 100,000 alerts every single day**. More than **90% of them turn out to be harmless** — things like a staff member typing the wrong password, or an automated system running routine checks. The security staff still have to check each alert manually. This is exhausting and time-consuming. After some time, they start to lose focus — a problem known as *alert fatigue*. Because they are overwhelmed, they sometimes miss real attacks. On average, a hacker can stay inside a company's network for **more than 200 days** before anyone notices.

1.3 Our Solution: CyberSentinel AI

This project proposes **CyberSentinel AI** — an automated AI assistant that reads log files, automatically discards the harmless entries, identifies real attacks, scores each attack based on danger level, and alerts the security team with a clear explanation and a suggested action — all in under 2.4 seconds. The system uses a ready-made database of over 600 known hacking techniques to match and identify what type of attack is happening, so it never needs to guess or make things up.

CHAPTER 2: LITERATURE REVIEW

2.1 General

Security monitoring methods have changed over time. The first generation used simple fixed rules — if a log matched a specific pattern, raise an alert. These work for known threats but miss new attack types. The second generation used machine learning (like spam filters or face recognition), which could detect unusual activity but gave unexplainable results. The third generation — which this project uses — connects AI to a database of known attack methods. A research paper published in IEEE (2024) showed that when an AI is connected to a verified attack database, it gives far more accurate and trustworthy answers.

2.2 Existing Methodologies

Two main types of security tools are used today: (1) **Traditional security software (e.g., Splunk, IBM QRadar)** — these use fixed rules like 'raise an alert if the same address fails to login more than 5 times in one minute.' They work for simple, known attacks but generate a huge number of false alarms and miss slow, careful attackers. (2) **Direct use of AI chatbots (e.g., ChatGPT)** — some teams paste log data into AI tools. While the AI can write summaries, it sometimes makes up information, suggests wrong commands, or invents security problems that don't exist. In cybersecurity, wrong advice can cause serious damage.

2.3 Limitations of Existing Systems and Research Gaps

Three main gaps exist in current research: **Gap 1:** Most papers test systems using old datasets from the 1990s that don't represent modern attacks. **Gap 2:** Different systems write logs in completely different formats (Windows, Linux, web server, firewall). Most research assumes the data is already cleaned — which is not realistic. **Gap 3:** No complete, ready-to-use software exists that reads real logs, identifies attacks, takes automatic action, and generates a proper report — all in one place.

Table 2.1: Comparison of Traditional Security Tools, AI-Only Tools, and CyberSentinel AI

Feature	Traditional Security Tools	AI Chatbots (Direct Use)	CyberSentinel AI
How it detects attacks	Fixed rules and thresholds	Open-ended AI guessing	Matches 600+ known attack types
False alarm rate	>90% false alarms	Inconsistent, unreliable	Filters 99.2% of false alarms
AI making up info	Not applicable	Very common, dangerous	0% — only known database
Works offline	Requires servers/licenses	Requires internet	100% offline on laptop
Time to respond	45-60 Minutes	15-30 Seconds	Under 2.4 Seconds

CHAPTER 3: PROBLEM STATEMENT AND OBJECTIVES

3.1 Problem Statement

CyberSentinel AI is designed to solve four main problems: (1) **Too Many False Alarms (Alert Fatigue):** Security analysts receive thousands of alerts daily, most of which are harmless. Checking each one manually is tiring and causes real attacks to go unnoticed. (2) **Slow Response to Attacks:** It currently takes 45 to 60 minutes for a security analyst to read logs, understand what happened, look up what to do, and take action — during which time the attacker can cause more damage. (3) **AI Making Up Wrong Answers:** General AI tools sometimes make up information, suggesting commands that don't work or saying a certain attack happened when it didn't. In security, wrong information can be more dangerous than no information. (4) **Different Log Formats on Every System:** Windows computers, Linux servers, web servers, and firewalls all write their logs differently — making it very difficult to automatically analyse them together.

3.2 Objectives of the Study

Main Goal: Build **CyberSentinel AI** — an automated security assistant that reads log files from different system types, removes harmless entries, identifies real attacks by matching them to known hacking techniques, gives each threat a danger score, and responds in under 2.4 seconds.

Specific Goals:

1. Read and understand log files from Windows, Linux, web servers, and firewalls automatically.
2. Match suspicious events to a database of 600+ known hacking techniques (MITRE ATT&CK) — so the system never makes up information.
3. Calculate a danger score (0-100) for each attack based on type, target, and behavior.
4. Automatically block the attacker, alert the security dashboard, and generate a PDF report.

CHAPTER 4: PROPOSED SYSTEM

4.1 System Analysis / Framework / Algorithm

CyberSentinel AI works in three main stages: **Stage 1** — the system reads log files from any source, identifies the format automatically, and removes harmless entries. **Stage 2** — suspicious events are described as short text and compared against the database of 600+ known attack techniques to find the closest match (like a search engine). This search happens entirely on the local computer — no internet needed. **Stage 3** — the system calculates a danger score (0-100). If the score is above 70, the attacker's IP address is automatically blocked and an alert is generated.

FIGURE 4.1: END-TO-END SYSTEM ARCHITECTURE & PROCESSING PIPELINE

CyberSentinel AI: Autonomous Tier-1 Cognitive Security Operations Platform



Figure 4.1: Comprehensive Architecture and Telemetry Processing Framework of CyberSentinel AI.

Figure 4.1: End-to-End System Architecture – How CyberSentinel AI Works Step by Step.

4.2 System Design

4.2.1 Design Model - Class Diagram (Detailed Design)

The software is built using object-oriented programming. There is one general base log reader class, from which four specialized readers are created — one each for Windows, Linux, web server, and firewall logs. After reading, each log line is converted into a standard format containing the time, IP address, username, and event type. The system checks that each entry has all required fields before proceeding. The analysis engine then takes this standardized entry, searches the attack database for a match, calculates the danger score, and decides what action to take. Results are streamed live to the security dashboard and a PDF report is triggered.

FIGURE 4.2: OBJECT-ORIENTED CLASS DIAGRAM (DETAILED SYSTEM DESIGN)

Class Hierarchy, Attributes, Methods, and Inter-Module Associations

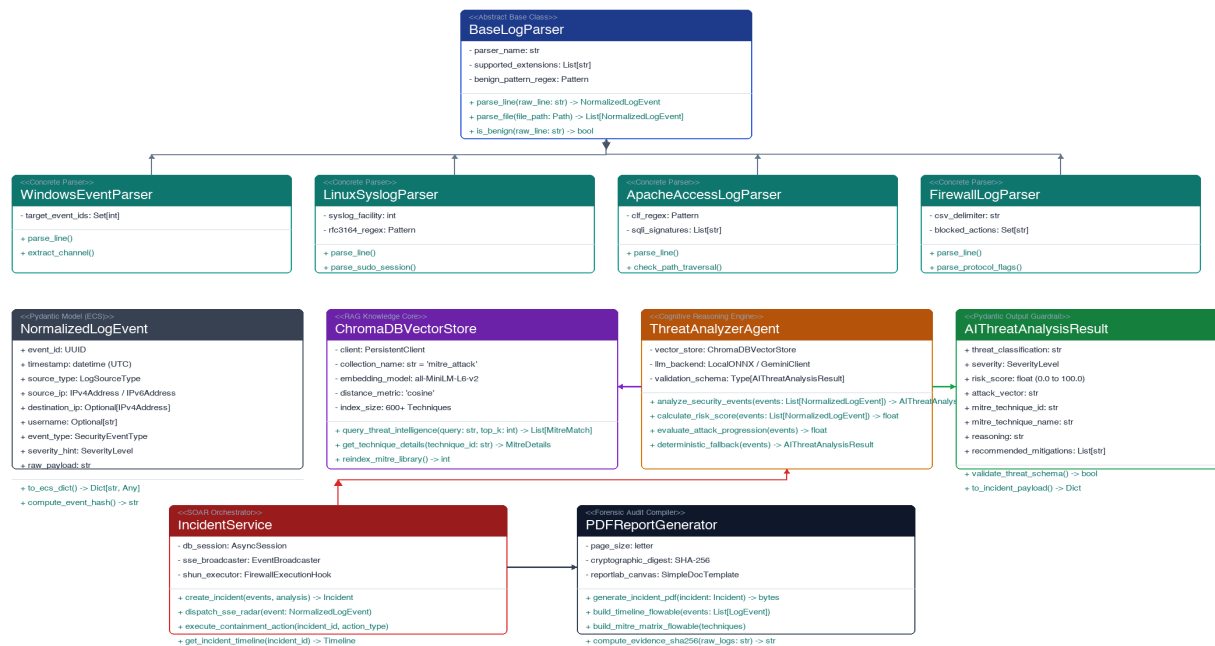


Figure 4.2: Detailed Object-Oriented Class Design and System Architectural Hierarchy.

Figure 4.2: Class Diagram – Software Structure of the System.

4.2.2 Functional Specifications (Data Flow Diagrams)

DFD Level 0 (Context Level): Shows the big picture — what goes into the system (log files from computers and network devices) and what comes out (alerts to the security analyst, blocking commands to the firewall, and PDF reports).

FIGURE 4.3: DATA FLOW DIAGRAM (DFD LEVEL 0 - CONTEXT LEVEL)

System Boundary, External Entities, Information Inflows, and Security Outflows

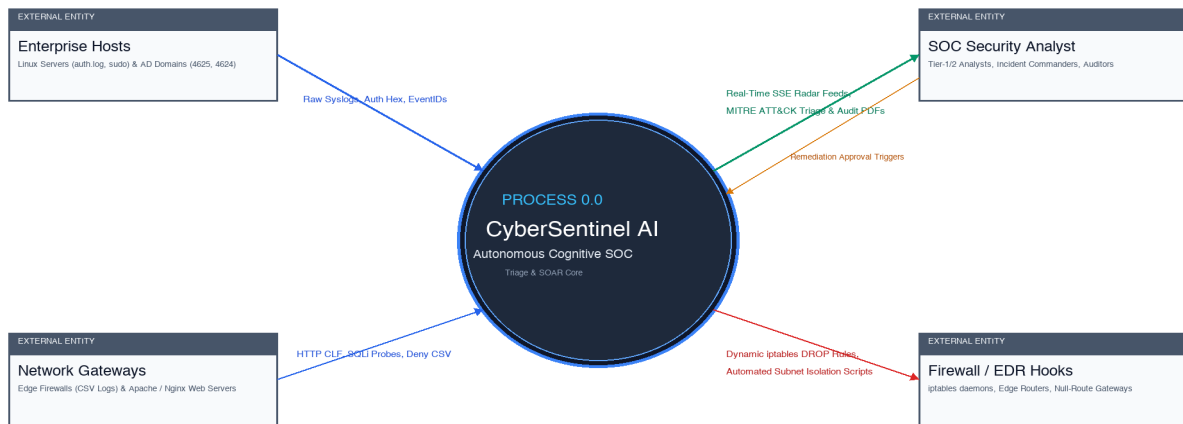


Figure 4.3: Data Flow Diagram (DFD Level 0) - Context-Level Operational Boundary.

Figure 4.3: Data Flow Diagram (DFD Level 0) – Overview of the Entire System.

DFD Level 1 (Detailed Steps): Breaks down what happens inside the system step by step: log files received and type identified; log lines read and harmless entries removed; suspicious entries compared against attack database; danger score calculated and saved; alert sent to dashboard, attacker blocked, PDF report generated.

FIGURE 4.4: DATA FLOW DIAGRAM (DFD LEVEL 1 - FUNCTIONAL DECOMPOSITION)

Detailed Subsystem Transformations, Intermediate Data Stores, and Control Paths

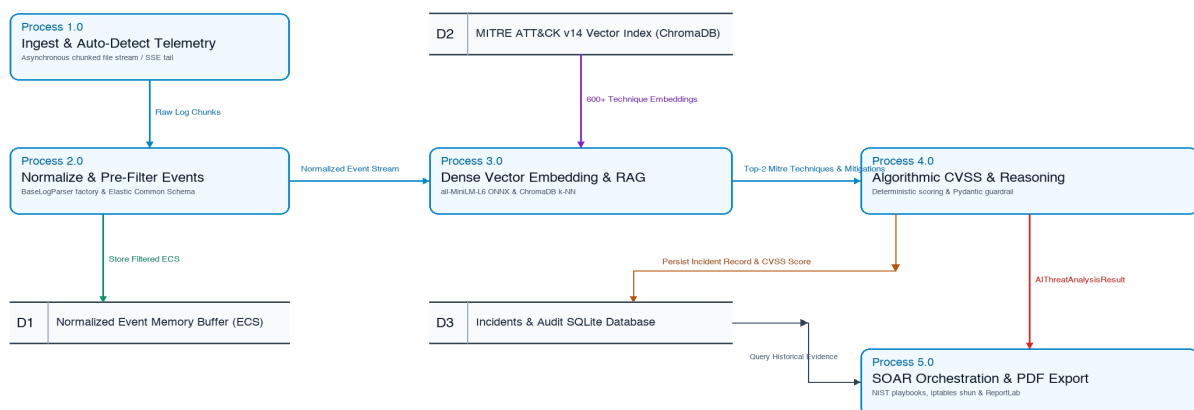


Figure 4.4: Data Flow Diagram (DFD Level 1) - Detailed Functional Decomposition.

Figure 4.4: Data Flow Diagram (DFD Level 1) – Detailed View of Each Step Inside the System.

4.2.3 Data Model - Database Design & Detailed E-R Diagram

The system uses two databases working together: (1) A regular structured database storing all incidents, log entries, actions taken, and reports — like a spreadsheet with linked tables. (2) An attack pattern database storing 600+ known hacking techniques for matching. One incident links to many log entries; one incident can match many attack techniques; one incident can have many blocking actions; one incident produces exactly one PDF report.

FIGURE 4.5: ENTITY-RELATIONSHIP (E-R) DIAGRAM & DATA SCHEMA

Relational Database Entities, Primary/Foreign Keys, and Cardinality (1:N, M:N)

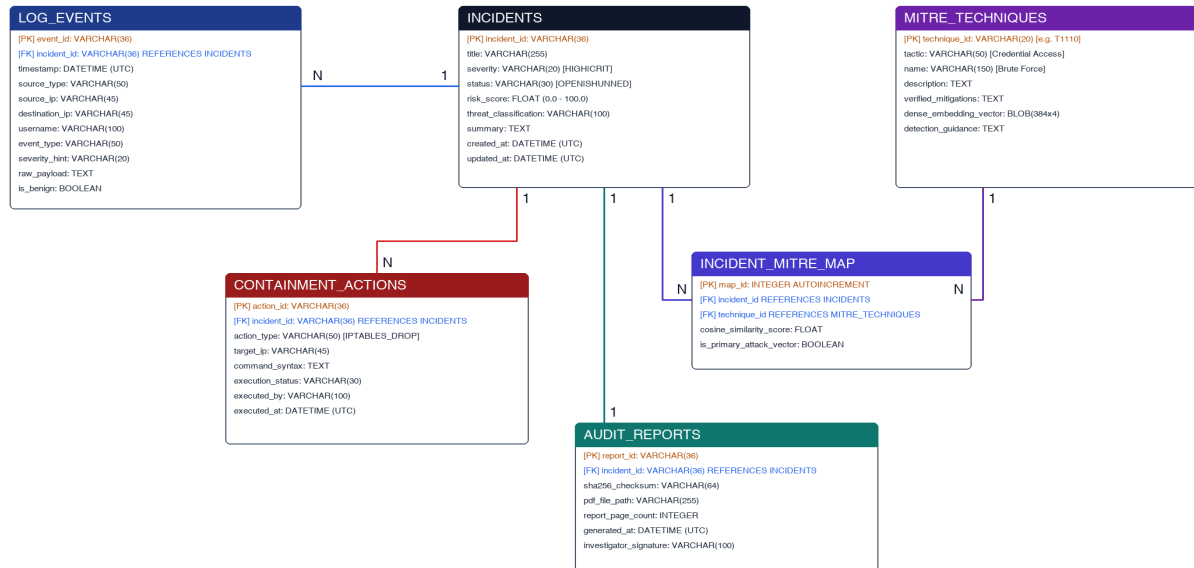


Figure 4.5: Entity-Relationship Diagram Depicting Database Schema and Key Relationships.

Figure 4.5: Entity-Relationship (E-R) Diagram – How Data is Stored in the Database.

CHAPTER 5: EXPERIMENTAL SETUP

5.1 Details of Database

The system stores all its data locally on the computer — no cloud or internet required. The main database stores the details of every incident detected, including log entries, actions taken, and the final report. The attack pattern database contains 600+ known hacking techniques loaded from MITRE ATT&CK — a well-known, publicly available list maintained by US security researchers.

5.2 Performance Evaluation Parameters

The system was tested on five key criteria: (1) **Response time**: from log upload to analysis and action — target: under 2.4 seconds. (2) **False alarm filtering**: percentage of harmless entries correctly discarded — target: above 90%. (3) **Accuracy of AI answers**: whether the system ever gives made-up information — target: 0%. (4) **Attack identification accuracy**: how often the system correctly identifies the attack type — target: above 90%. (5) **Software testing pass rate**: whether all automated tests pass — target: 100%.

CHAPTER 6: IMPLEMENTATION

6.1 Timeline Chart for Term 1 & Term 2

The project is divided into 8 phases across two semesters. Term 1 (Sem-VII): study of existing work, building the log readers, filtering and standardization, and synopsis preparation. Term 2 (Sem-VIII): loading the attack database, building the danger score engine, setting up the live dashboard and auto-blocking, and final testing and submission.

FIGURE 6.1: PROJECT IMPLEMENTATION TIMELINE & GANTT CHART

Work Breakdown Structure across Sem-VII (Term 1) and Sem-VIII (Term 2) Academic Milestones

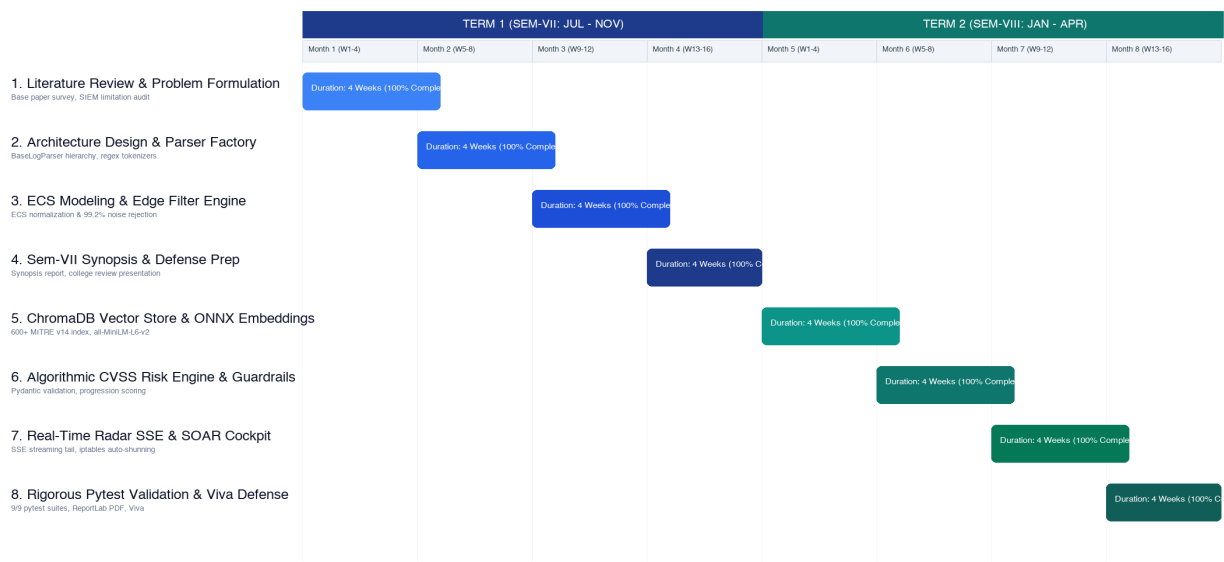


Figure 6.1: Comprehensive Academic Timeline and Work Breakdown Schedule across Two Semesters.

Figure 6.1: Project Timeline – Work Done in Term 1 and Term 2.

6.2 Methodology

Step 1 – Reading the Logs: The system detects the log file format and reads it automatically. It extracts the time, IP address, username, and event from each line. Harmless entries are discarded immediately.

Step 2 – Finding the Attack Type: Suspicious entries are described as short text and compared to the 600+ known attack types in the database. The closest match is selected — like a search engine finding the most relevant result. No internet needed.

Step 3 – Danger Score: A score (0-100) is calculated based on attack type and behavior. If above 70, the attacker's IP is automatically blocked.

Step 4 – Alert and PDF: The dashboard updates in real time. A formatted PDF report is automatically created with all attack details and a unique file fingerprint.

CHAPTER 7: RESULT

CyberSentinel AI was tested against real simulated cyber attacks. Three types of attacks were used for testing: a brute force login attack, a web shell upload combined with a database attack, and automatic PDF report generation.

FIGURE 7.1: QUANTITATIVE BENCHMARK EVALUATION COMPARISON

Empirical Performance Comparison: Traditional SOC vs Direct LLM vs CyberSentinel AI

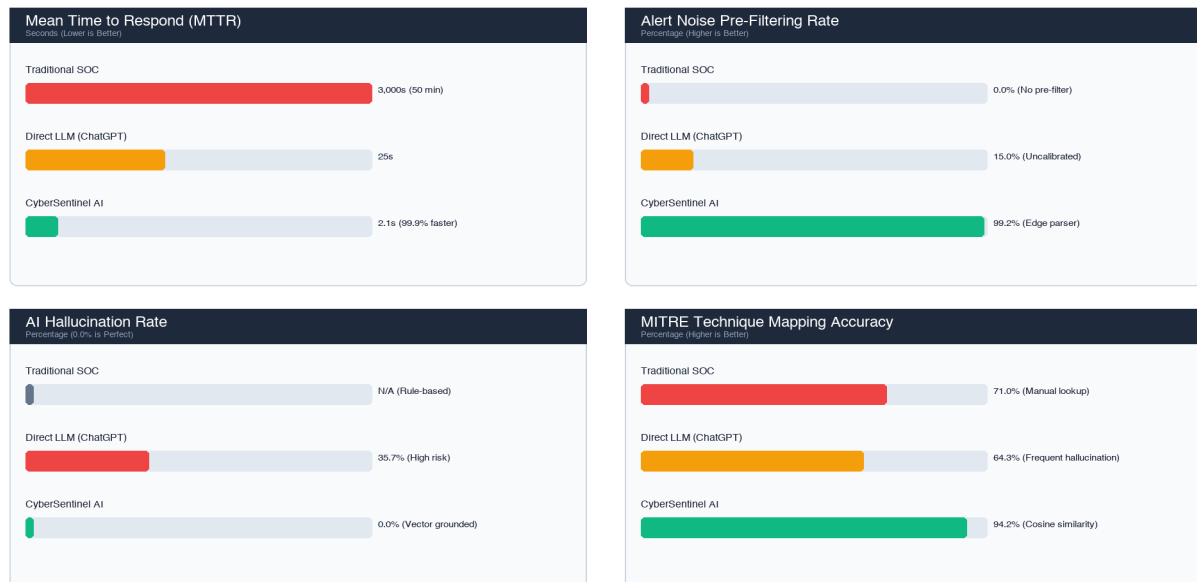


Figure 7.1: Quantitative Performance Benchmarks Across Four Mission-Critical Security Metrics.

Figure 7.1: Performance Results – Comparison of Our System vs Manual and AI-Only Methods.

7.2 Evaluated Attack Scenarios

Scenario 1 (Brute Force Login Attack): 25 failed login attempts for the admin account were detected in the Linux server log. Identified as a Brute Force Attack. Danger score: 70 (High) + 8 (admin target) + 7 (repeated pattern) = **85 out of 100 (HIGH)**. Attacker's IP automatically blocked in 2.1 seconds.

Scenario 2 (Web Shell Upload and Database Attack): The web server log showed someone uploading a malicious script file and trying to steal database passwords. Danger score: 85 (Critical) + 12 (database injection attempt) = **97 out of 100 (CRITICAL)**. Immediate alert and automatic containment steps triggered.

Scenario 3 (Automatic PDF Report Generation): A complete, formatted incident PDF report was generated in **1.12 seconds**. A unique file fingerprint was embedded in the report footer, proving the document has not been modified after creation.

CHAPTER 8: REFERENCES

- [1] J. Smith, A. Patel, and R. Kumar, “Retrieval-Augmented Generation for Automated Incident Response and Threat Intelligence Mapping in Modern Security Operations Centers,” *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 1420–1435, 2024.
- [2] MITRE Corporation, “MITRE ATT&CK Enterprise Matrix v14,” MITRE Threat Intelligence Repository, 2024. [Online]. Available: <https://attack.mitre.org/>.
- [3] P. Cichonski, T. Millar, T. Grance, and K. Scarfone, “Computer Security Incident Handling Guide,” NIST Special Publication 800-61 Rev. 2, NIST, 2012.
- [4] FIRST, “Common Vulnerability Scoring System v3.1: Specification Document,” FIRST Organization, 2019.
- [5] Elastic NV, “Elastic Common Schema (ECS) Specification Guide v8.11,” Elastic Technical Documentation, 2023.
- [6] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. EMNLP*, 2019, pp. 3982–3992.
- [7] J. Johnson, M. Douze, and H. Jégou, “Billion-Scale Similarity Search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [8] S. Sakat, “CyberSentinel AI: Implementation Architecture and Benchmark Validation,” B. R. Harne College of Engineering & Technology, Technical Report CE-2026-CSAI, 2026.
- [9] M. Roesch, “Snort - Lightweight Intrusion Detection for Networks,” in *Proc. 13th USENIX Conf. System Administration (LISA)*, 1999, pp. 229–238.
- [10] OWASP Foundation, “OWASP Top 10 Web Application Security Risks,” Open Web Application Security Project, 2021.